IT342 – System Integration and Architecture

MedBuddy

Phase 1 Deliverable

User Registration and Login

Prepared by	Aquino, Julianne Rose P.
Section	IT342-G2
Project Name	MedBuddy
Date	March 9, 2026
Phase	Phase 1 – Authentication
Status	Completed

1. Final Commit Information

Commit Message

```
IT342 Phase 1 – User Registration and Login Completed
```

Repository Details

Repository Name: IT342_MedBuddy_Aquino

Branch: main

Commit Hash: *6a31fd3693f720995c03340187e97792078d1ad5*

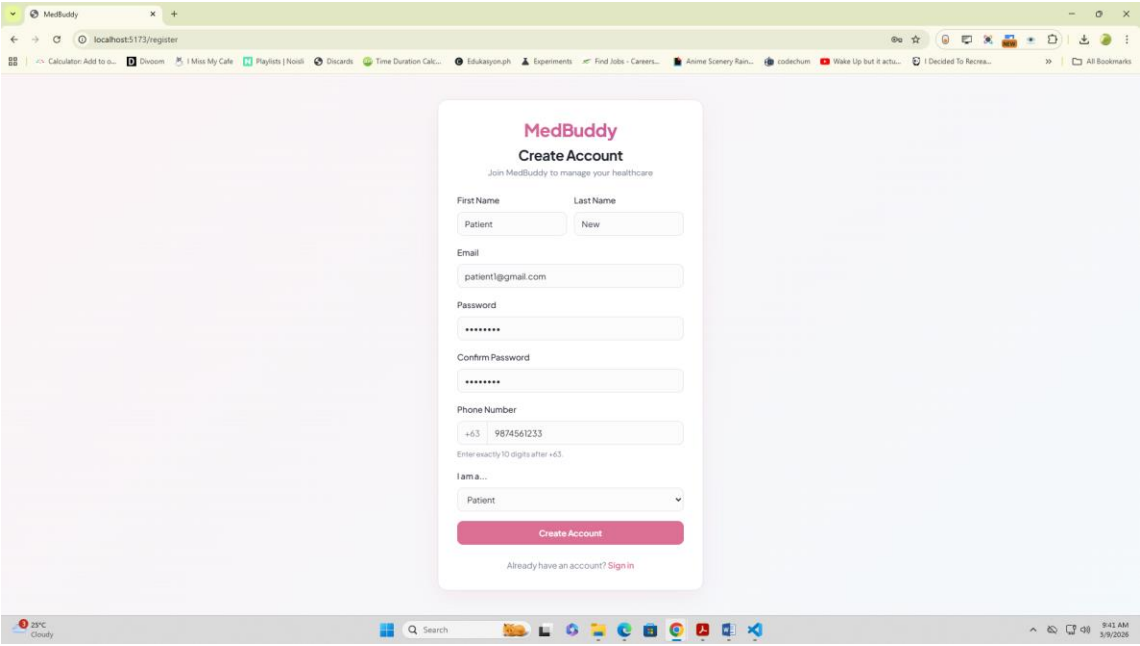
Commit Link: https://github.com/oblivjules/IT342_MedBuddy_G2_Aquino/commit/6a31fd3693f720995c03340187e97792078d1ad5

Repository Link: https://github.com/oblivjules/IT342_MedBuddy_Aquino.git

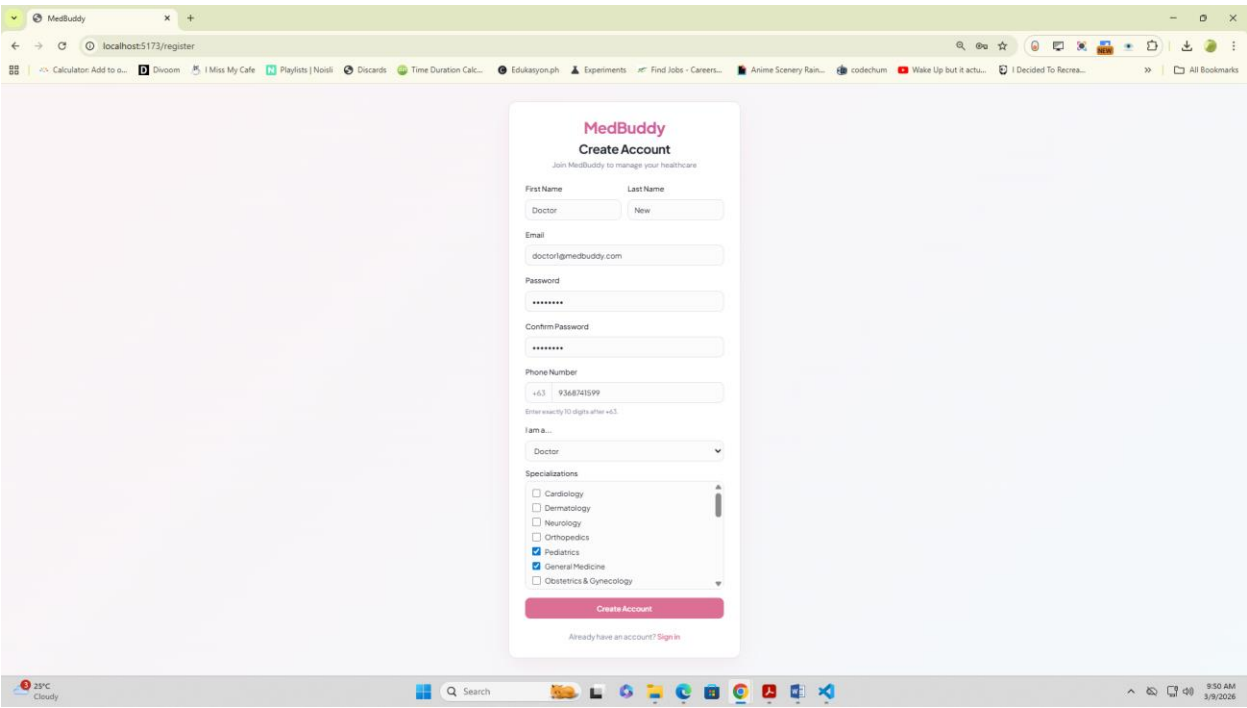
2. Screenshots of Implementation

Screenshot 1 – Registration Page

a. Patient Registration

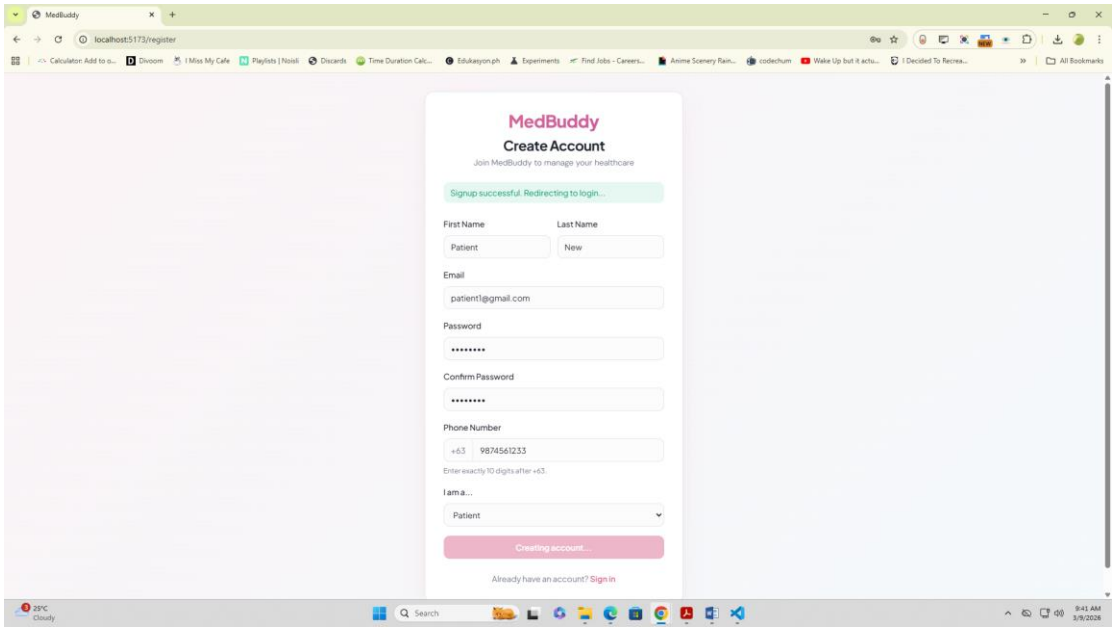


b. Doctor Registration

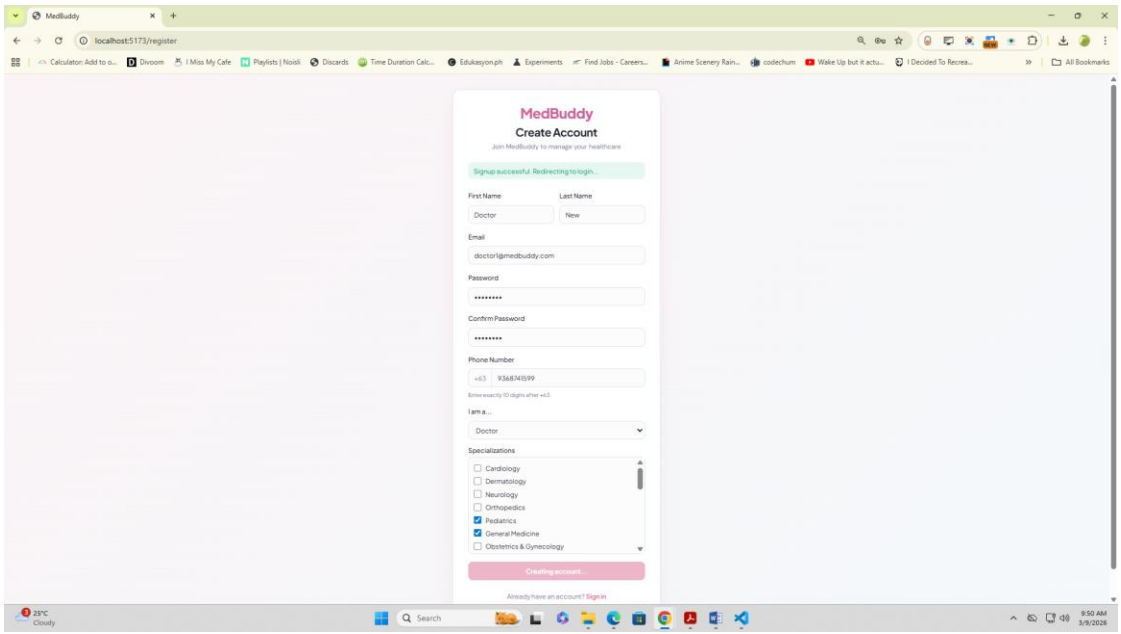


Screenshot 2 – Successful User Registration

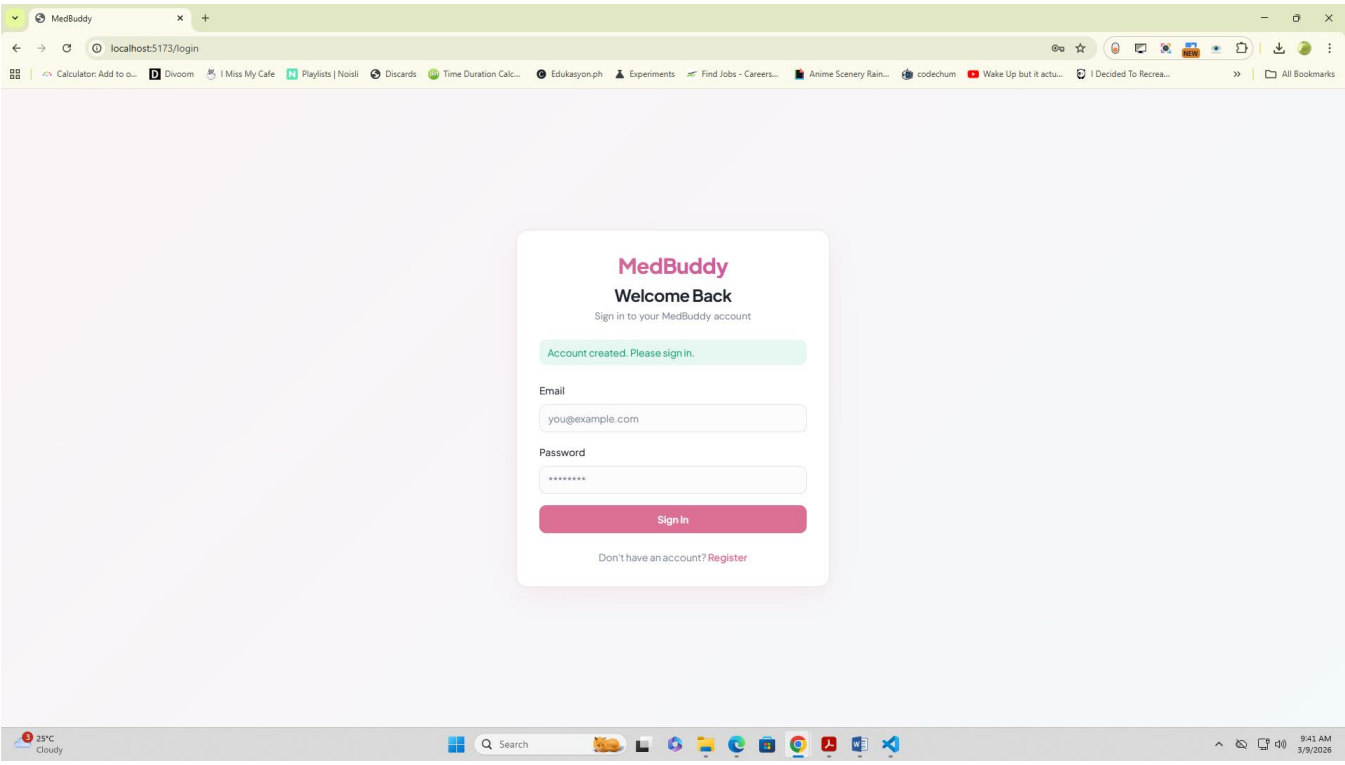
a. Patient



b. Doctor

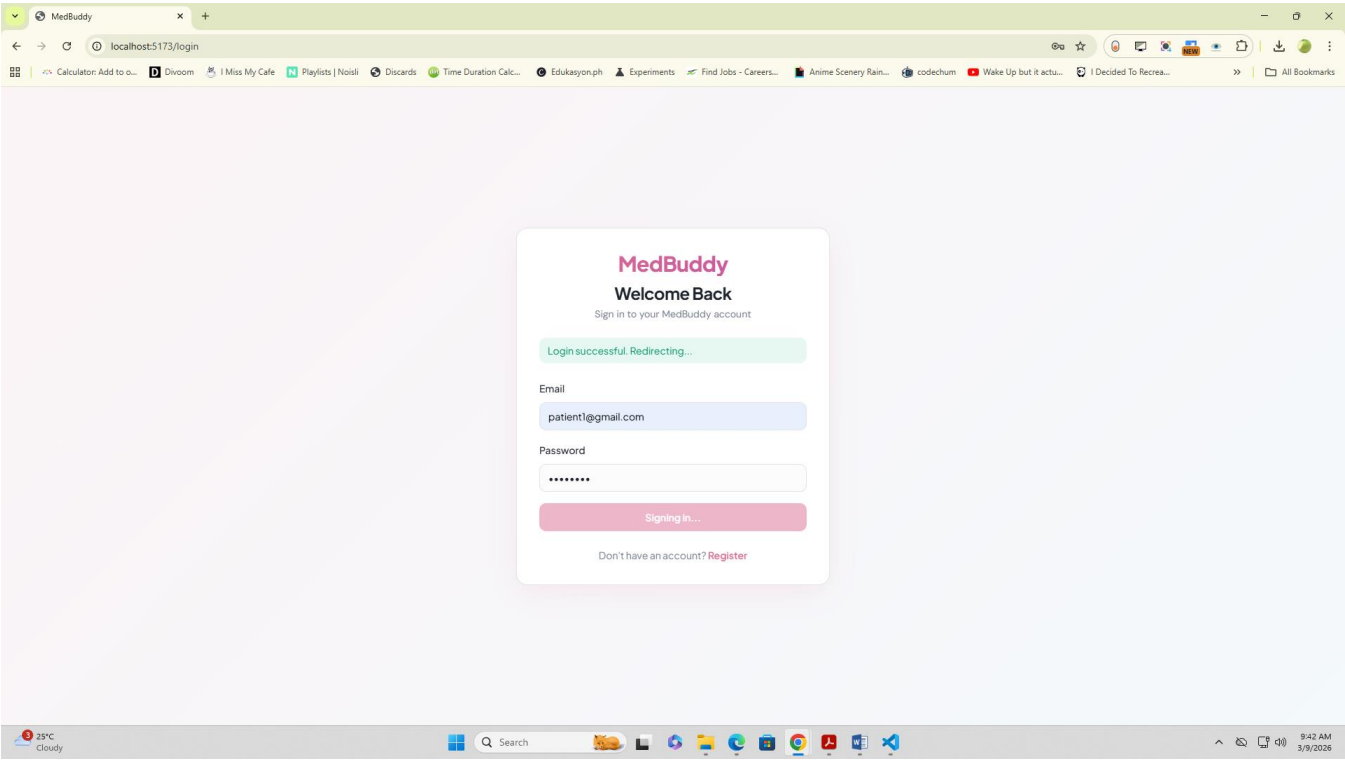


Screenshot 3 – Login Page (Same Login Page for both Roles)

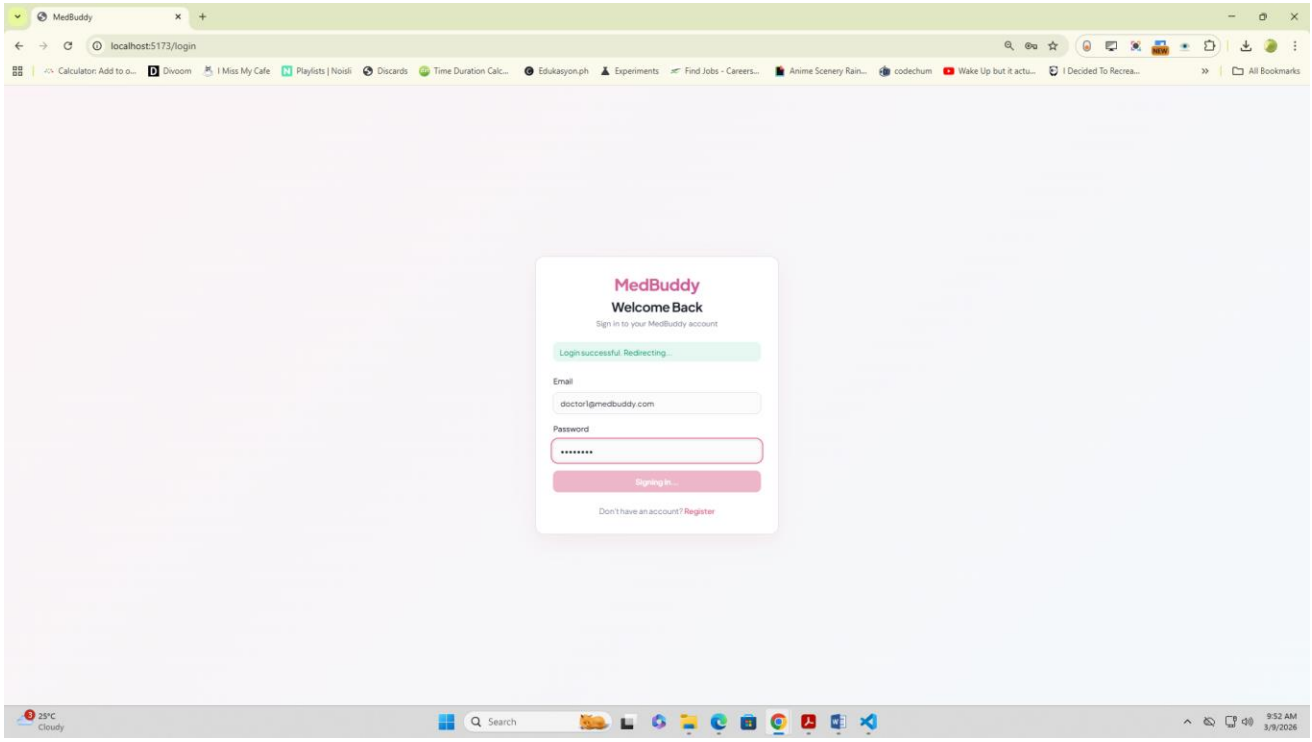


Screenshot 4 – Successful Login

a. Patient

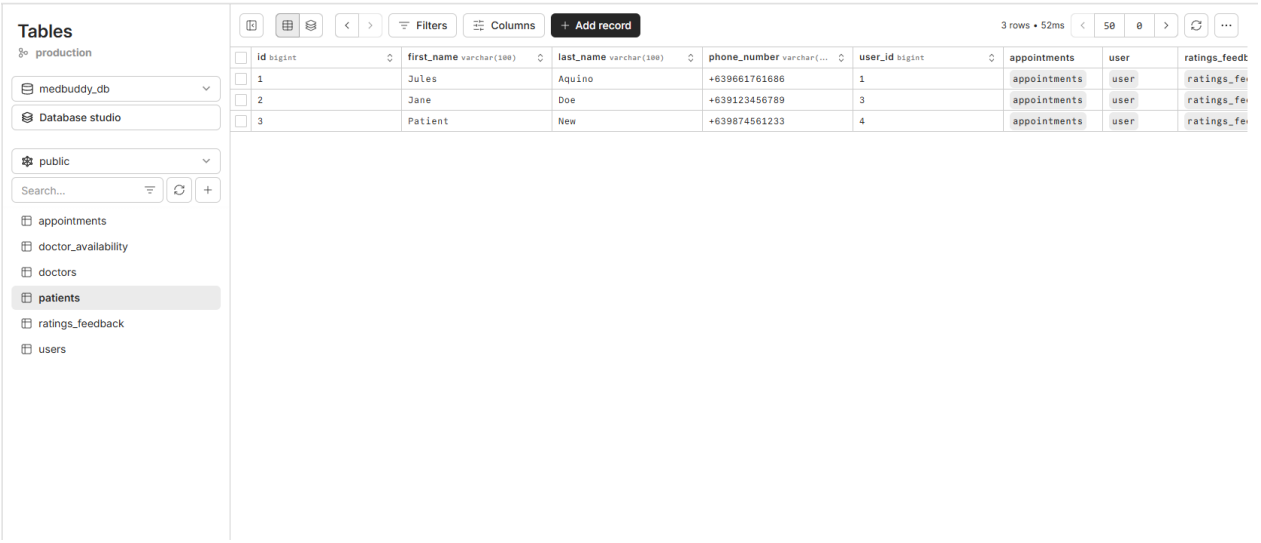


b. Doctor



Screenshot 5 – Database Record of Registered User

a. New Patient Record



b. New Doctor Record

Tables
production
medbuddy_db
Database studio
public
Search...
appointments
doctor_availability
doctors
patients
ratings_feedback
users

Filters | Columns | + Add record | 2 rows • 64ms | 50 | 0

id bigint	first_name varchar(100)	last_name varchar(100)	phone_number varchar(...	specialization varchar(2...	user_id bigint	appointments
1	new	user	+639661761686	Cardiology, Orthopedics	2	appointments
2	Doctor	New	+639368741599	General Medicine, Pedi…	5	appointments

c. Both records showing on the Users Table

Tables
production
medbuddy_db
Database studio
public
Search...
appointments
doctor_availability
doctors
patients
ratings_feedback
users

Filters | Columns | + Add record | 5 rows • 53ms | 50 | 0

id bigint	created_at timestamp	email varchar(255)	password_hash varchar...	provider varchar(20)	role varchar(20)	doctors	pa
1	2026-03-09 09:25:31.11…	aquinojulianne.r@gmail…	$2a$10$go0OwumWGBhzHFY…	LOCAL	PATIENT	doctors	pa
2	2026-03-09 09:37:53.84…	newuser@gmail.com	$2a$10$SBAldCBnra/M3Qg…	LOCAL	DOCTOR	doctors	pa
3	2026-03-09 09:38:54.60…	janedoe@gmail.com	$2a$10$qdDXBtpwUGPgNdo…	LOCAL	PATIENT	doctors	pa
4	2026-03-09 09:41:36.98…	patient1@gmail.com	$2a$10$NZpw7voz8HyLq5F…	LOCAL	PATIENT	doctors	pa
5	2026-03-09 09:50:42.31…	doctor1@medbuddy.com	$2a$10$TCW58yvc4FHDhwU…	LOCAL	DOCTOR	doctors	pa

3. Implementation Summary

3.1 User Registration

Registration Fields Used

The backend registration request (RegisterRequest) accepts:

- firstName - required
- lastName - required
- email - required, unique, used as login identifier
- password - required, minimum 8 characters
- role - required, must be PATIENT or DOCTOR
- phoneNumber - optional at backend, but if provided must match +63 followed by exactly 10 digits
- specialization - optional in DTO, but required by service logic when role=DOCTOR

Validation Process

Server-side validation is enforced with Jakarta Bean Validation plus service-level checks:

- @Email validates email format
- @Size(min = 8) validates password length
- @NotBlank validates required text fields
- @NotNull validates role presence
- role must map to enum values PATIENT or DOCTOR
- If role is DOCTOR, UserService.register() throws IllegalArgumentException when specialization is missing/blank
- Validation errors return HTTP 400 using ProblemDetail with an errors object for field-level messages

How Duplicate Accounts are Prevented

- UserService.register() checks userRepository.existsByEmail(email) before insert
- If email already exists, it throws IllegalStateException
- Global exception handler maps this to HTTP 409 Conflict
- Database also has unique constraint on users.email (@Column(unique = true)), providing a second layer of protection
- There is no custom DuplicateEmailException or AUTH-DUPLICATE-EMAIL code in current implementation

How Passwords are Stored Securely

- Passwords are hashed with Spring Security BCryptPasswordEncoder (default strength)
- Plaintext password is never stored in DB
- Hash is stored in users.password_hash (User.passwordHash)
- API responses use DTOs (AuthResponse, UserDto) that do not include password hash
- passwordHash is nullable to support non-local providers (e.g., Google OAuth users)

3.2 User Login

Login Credentials Used

Users submit email and password JSON to POST /api/auth/login

How the System Verifies Users

The login flow works as follows:

- Controller receives request and calls UserService.login()
- AuthenticationManager.authenticate(...) validates credentials via DaoAuthenticationProvider
- UserDetailsServiceImpl.loadUserByUsername(email) loads user and hashed password
- BCrypt comparison is handled by Spring Security's authentication provider
- Invalid credentials trigger BadCredentialsException, returned as HTTP 401 with detail "Invalid email or password"

What Happens After Successful Login

Upon successful authentication, the system performs the following:

- Backend loads user and builds AuthResponse
- JwtUtil.generateToken(email) creates a signed JWT with:
  - subject = email
  - issued-at timestamp
  - expiration timestamp
- Response payload is:
  - { token, user }
- user includes profile-enriched fields from UserDto:
  - id, email, role, profileId, firstName, lastName, phoneNumber, specialization (doctor only)

Web Client Behavior (Current Project)

- React stores token in localStorage key medbuddy_token
- React stores user in localStorage key medbuddy_user
- Axios request interceptor adds Authorization: Bearer <token>
- Axios response interceptor handles 401 by clearing auth storage and redirecting to /login
- Login page redirects to:
  - /doctor/dashboard for DOCTOR
  - /patient/dashboard for PATIENT
- Register page does not auto-login; it redirects to login after successful signup

3.3 Database Table

The primary table used for authentication is the users table, stored in a PostgreSQL database hosted on Neon.

Column	Type	Description
`id`	`BIGINT identity PK`	Auto-incrementing primary key
`email`	`VARCHAR(255) UNIQUE NOT NULL`	Unique login identifier
`password_hash`	`string column (nullable)`	BCrypt hash for local auth; nullable for OAuth users
`role`	`enum stored as string (VARCHAR(20))`	Either PATIENT or DOCTOR
`provider`	`enum stored as string (VARCHAR(20))`	LOCAL for email/password; GOOGLE for OAuth2 users
`created_at`	`timestamp`	Auto-set via @CreationTimestamp

Related profile tables:

- patients with one-to-one unique user_id FK to users
- doctors with one-to-one unique user_id FK to users
- These hold role-specific profile fields (firstName, lastName, phoneNumber, specialization)

3.4 API Endpoints

Method	Endpoint	Description
POST	`/api/auth/register`	Register user and profile; returns 201 with { token, user }
POST	`/api/auth/login`	Authenticate user; returns 200 with { token, user }
GET	`/api/users/me`	Authenticated current-user profile (UserDto
GET	`/api/auth/health`	Public health check; returns 200 with { "status": "UP" }
GET	`/api/users/doctors`	Authenticated list of doctors (DoctorDto[])

Notes:

- There is no /api/auth/me endpoint in the current backend.
- Protected routes are enforced by SecurityConfig (/api/auth/** is public; other routes require authentication).

Request & Response Examples

POST /api/auth/register — Request Body:

```
{
  "firstName": "Juan",
  "lastName": "Dela Cruz",
  "email": "juan@example.com",
  "password": "SecurePass123",
  "role": "PATIENT",
  "phoneNumber": "+639171234567"
}
```

POST /api/auth/register — Success Response (201):

```
{

  "token": "eyJhbGciOiJIUzI1NiJ9...",

  "user": {

    "id": 1,

    "email": "juan@example.com",

    "role": "PATIENT",

    "profileId": 10,

    "firstName": "Juan",

    "lastName": "Dela Cruz",

    "phoneNumber": "+639171234567",

    "specialization": null

  }

}
```

POST /api/auth/login – Request

```
{

  "email": "juan@example.com",

  "password": "SecurePass123"

}
```

POST /api/auth/login - Success Response (200)


```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "user": {
    "id": 1,
    "email": "juan@example.com",
    "role": "PATIENT",
    "profileId": 10,
    "firstName": "Juan",
    "lastName": "Dela Cruz",
    "phoneNumber": "+639171234567",
    "specialization": null
  }
}
```